



SuperAhorro

Aplicación Android de Control de Gastos en Supermercados

Kotlin • Jetpack Compose • MVVM • Navigation • SharedPreferences

Integrantes:

Calvo Tomas
Benjamin Alasino

Tecnologías y Lenguajes

Stack completo del proyecto



Kotlin

Lenguaje oficial de Android
Null-safety, corrutinas integradas
Menos código que Java
Interoperable con Java 100%



Gradle (KTS)

build.gradle.kts — sintaxis Kotlin
libs.versions.toml — catálogo de versiones
AGP 9.1.1 — Android Gradle Plugin
Compila el APK final



Jetpack Compose

UI declarativa 100%
@Composable functions
Material 3 (Material You)
Sin XML layouts



Android Studio

IDE oficial de Google
API 36 (Android 15)
Min SDK 24 (Android 7.0)
Emulador y debugging integrado



SharedPreferences

Persistencia local de datos
JSON serializado con org.json
Datos aislados por usuario
No requiere servidor



Navigation Compose

navigation-compose:2.7.7
NavHost + composable()
Argumentos tipados (IntType)
Back-stack automático

Requerimientos

Stack completo del proyecto

Pantalla de bienvenida/Splash
con navegación automática

Flujo de registro e inicio de
sesión

Gestión de perfil con edición
de datos

Registro de compras
(supermercado, fecha, hora,
total)

Gestión de productos
asociados a compras

Listado de últimas compras

Historial completo de
compras

Estadísticas con gráficos
visuales

Navegación intuitiva entre
todas las pantallas

Requisitos No Funcionales — Checklist

Todos los requisitos del proyecto verificados en el código



Proyecto creado desde cero en Android Studio



UI usable, clara y visualmente consistente



Jetpack Compose



Corrutinas en operaciones asincrónicas



Versionado en GitHub

<https://github.com/tomas-calvo01/SuperAhorro>



Nombre de paquete: `com.undef.nombreappalumno`

`namespace = "com.undef.superahorroCalvoAlasino"`



Navegación entre pantallas

`NavHost` + 12 rutas (`NavRoutes` sealed class) + argumentos tipados

Estructura del Proyecto

Organización de carpetas y paquete com.undef.*

```
app/src/main/java/com/undef/superahorroCalvoAlasino/  
├── MainActivity.kt  
├── navigation/  
│   ├── AppNavGraph.kt  
│   └── NavRoutes.kt  
├── ui/  
│   ├── screens/  
│   │   ├── SplashScreen.kt  
│   │   ├── LoginScreen.kt  
│   │   ├── RegistroScreen.kt  
│   │   ├── HomeScreen.kt  
│   │   ├── NuevaCompraScreen.kt  
│   │   ├── NuevoProductoScreen.kt  
│   │   ├── DetalleCompraScreen.kt  
│   │   ├── HistorialScreen.kt  
│   │   ├── EstadisticasScreen.kt  
│   │   ├── PerfilScreen.kt  
│   │   ├── EditarPerfilScreen.kt  
│   │   └── SettingsScreen.kt  
│   ├── components/  
│   │   └── BottomNavBar.kt  
│   ├── theme/  
│   │   ├── Color.kt  
│   │   ├── Theme.kt  
│   │   └── Type.kt  
├── viewmodel/  
│   ├── CompraViewModel.kt  
│   └── UsuarioViewModel.kt  
└── model/  
    └── Models.kt
```

Nombre

```
// app/build.gradle.kts  
namespace = "com.undef.superahorroCalvoAlasino"  
applicationId = "com.undef.superahorroCalvoAlasino"
```

model/

Data classes: Compra, Producto, Usuario

viewmodel/

UsuarioViewModel, CompraViewModel

Almacena el estado y la lógica de negocio de la app. Expone datos observables (como listas de compras) que los Composables pueden escuchar

ui/screens/

12 pantallas @Composable

navigation/

NavRoutes (Define todas las rutas (pantallas) de la app)

AppNavGraph (Es el NavHost que orquesta la navegación entre esas rutas)

Jetpack Compose

UI declarativa — MainActivity.kt como punto de entrada

```
// MainActivity.kt
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            SuperAhorroTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    AppNavGraph(applicationContext)
                }
            }
        }
    }
}
```

¿Por qué Jetpack Compose?

setContent { }

Es la función que define toda la UI de la Activity usando Compose

@Composable functions

Cada pantalla es una función anotada con @Composable que describe la UI.

SuperAhorroTheme { }

Aplica:
colores, tipografía y formas definidos en
ui/theme/Theme.kt

AppNavGraph()

Carga todas las 12 pantallas y su navegación

Navegación — appnevGrap

```
@Composable
fun AppNavGraph(context: Context? = null) {
    val navController = rememberNavController()
    val compraViewModel = remember {
        CompraViewModel(context)
    }
    val usuarioViewModel = remember {
        UsuarioViewModel(context)
    }

    NavHost(
        navController = navController,
        startDestination = NavRoutes.Splash.route
    ) {
        composable(NavRoutes.Splash.route) { SplashScreen(navController) }
        composable(NavRoutes.Login.route) { LoginScreen(navController, usuarioViewModel, compraViewModel) }
        composable(NavRoutes.Registro.route) { RegistroScreen(navController, usuarioViewModel, compraViewModel) }
        composable(NavRoutes.Home.route) { HomeScreen(navController, compraViewModel) }
        composable(NavRoutes.Historial.route) { HistorialScreen(navController, compraViewModel) }
        composable(NavRoutes.Estadisticas.route) { EstadisticasScreen(navController, compraViewModel) }
        composable(NavRoutes.Perfil.route) { PerfilScreen(navController, usuarioViewModel, compraViewModel) }
        composable(NavRoutes.EditarPerfil.route) { EditarPerfilScreen(navController, usuarioViewModel) }
        composable(NavRoutes.Settings.route) { SettingsScreen(navController) }
    }
}
```

¿Por qué sealed class?

Una sealed class en Kotlin garantiza que SOLO existen los subtipos declarados en el archivo. El compilador verifica exhaustividad (when expressions).

Splash	Verifica si hay usuario
Login / Registro	Autenticación
Home	Dashboard principal
Historial / Estadísticas	Vistas de datos
Detalle Compra	withId(compralId)
Nuevo Producto	Argumento tipado Int

Pantalla Login — LoginScreen.kt

ui/screens/LoginScreen.kt · Estado local + validación + navegación

```
@Composable
fun LoginScreen(navController: NavController,
               usuarioVM: UsuarioViewModel,
               compraVM: CompraViewModel) {

    // Estado local de la pantalla
    var email    by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var error    by remember { mutableStateOf("") }

    OutlinedTextField(
        value = email,
        onChange = { email = it; error = "" },
        label = { Text("Email") },
        keyboardOptions = KeyboardOptions(
            keyboardType = TextInputType.Email
        )
    )

    OutlinedTextField(
        value = password,
        onChange = { password = it },
        visualTransformation = PasswordVisualTransformation()
    )

    Button(onClick = {
        if (usuarioVM.validarCredenciales(email, password)) {
            compraVM.cargarComprasDelUsuario(email)
            navController.navigate(NavRoutes.Home.route) {
                popUpTo(NavRoutes.Login.route) { inclusive = true }
            }
        } else { error = "Credenciales incorrectas" }
    }) { Text("Iniciar sesión") }
}
```



Super Ahorro

Email o contraseña incorrectos

Iniciar sesión

¿No tenés cuenta? [Registrate](#)

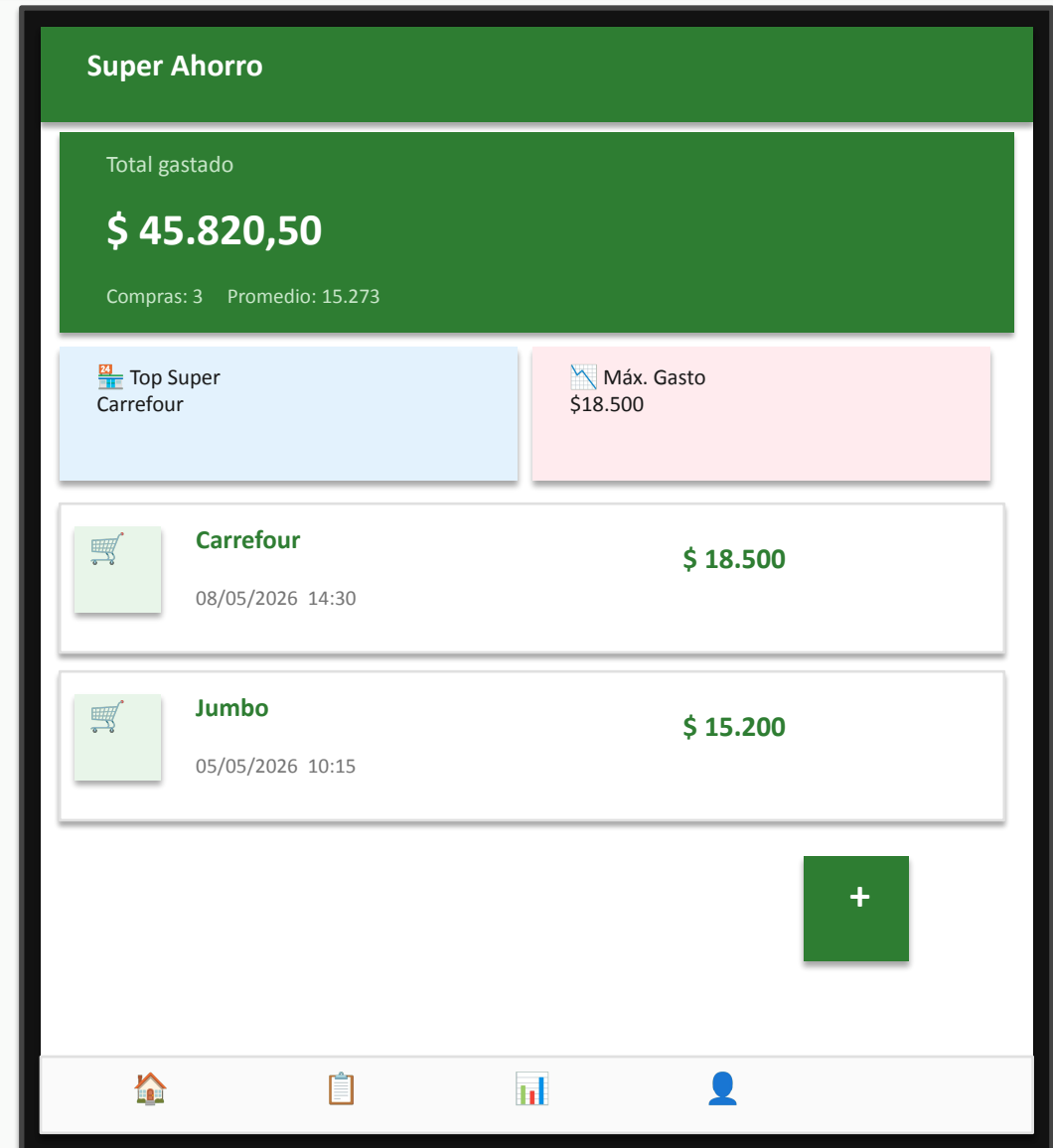
Pantalla Home — HomeScreen.kt

ui/screens/HomeScreen.kt · Scaffold + LazyColumn + Cards

```
@Composable
fun HomeScreen(navController: NavController,
               compraViewModel: CompraViewModel) {

    val compras      = compraViewModel.compras
    val totalGastado = compraViewModel.calcularTotalGastado()

    Scaffold(
        topBar = { TopAppBar(title = { Text("Super Ahorro") },
                              colors = TopAppBarDefaults.topAppBarColors(
                                  containerColor = Color(0xFF2E7D32),
                                  titleContentColor = Color.White)) },
        bottomBar = { BottomNavBar(navController) },
        floatingActionButton = {
            FloatingActionButton(
                onClick = { navController.navigate(NavRoutes.NuevaCompra.route) },
                containerColor = Color(0xFF2E7D32)
            ) { Icon(Icons.Default.Add, null) }
        }
    ) { padding ->
        LazyColumn(...) {
            item { /* Tarjeta Total Gastado */ }
            item { /* Insight Cards: Top Super / Máx Gasto */ }
            items(compras) { compra ->
                CompraCard(compra, onClick = {
                    navController.navigate(
                        NavRoutes.DetalleCompra.withId(compra.id)
                    )
                })
            }
        }
    }
}
```



Estadísticas — Gráfico Circular Propio

ui/screens/EstadisticasScreen.kt · Custom Canvas @Composable

```
@Composable
fun PieChart(data: List<Pair<String, Double>>,
             totalGastado: Double, modifier: Modifier) {

    val total = data.sumOf { it.second }
    val colors = data.indices.map { getColorForIndex(it) }

    Box(modifier, contentAlignment = Alignment.Center) {
        Canvas(modifier = Modifier.fillMaxSize()) {
            val radius = size.minDimension / 2.3f
            var currentAngle = -90f

            data.forEachIndexed { index, (_, value) ->
                val sweep = (value / total * 360f).toFloat()

                drawArc(color = colors[index],
                       startAngle = currentAngle,
                       sweepAngle = sweep, useCenter = true, ...)

                currentAngle += sweep
            }
            // Círculo blanco central = efecto donut
            drawCircle(Color.White, radius * 0.6f, ...)
        }
        // Texto total en el centro del donut
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Text("Total"); Text("$ $ {total}")
        }
    }
}
```

Estadísticas

Total Gastado: \$ 45.820,50 (3 compras)



\$ 45.820

-  Carrefour 40%
-  Jumbo 33%
-  Día 27%

SuperAhorro

Kotlin · Jetpack Compose · MVVM · Navigation Compose

- ▶ github: <https://github.com/tomas-calvo01/SuperAhorro>
 - ▶ 12 rutas de navegación con sealed class
 - ▶ 2 ViewModels con estado reactivo
- ▶ Persistencia JSON por usuario con SharedPreferences
- ▶ Gráfico circular implementado con Canvas API

com.undef.superahorroCalvoAlasino · API 36 (Android 15)